

Preparación para Despliegue Local o en la Nube (Render)

Actividad N°: 36 **Fecha:** 20/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Dejar lista la configuración de despliegue (`render.yaml`) para publicar backend y frontend, resolviendo antes que nada el hallazgo de seguridad pendiente desde la Semana 2/7 (credenciales de MongoDB en texto plano dentro del repositorio), y complementar la preparación con un archivo de referencia de variables de entorno, una guía paso a paso para rotar las credenciales comprometidas en MongoDB Atlas, y una revisión de seguridad final del repositorio completo.

2. Corrección crítica aplicada: credenciales expuestas en `render.yaml`

Archivo modificado: `render.yaml`

```
- key: MONGODB_URI
- value:
  mongodb+srv://gabriel:gabriel@bddshakira.l08bhec.mongodb.net/Shakira8Noviembre
+ sync: false # Configurar manualmente en el dashboard de Render (no
  versionar credenciales)
```

`sync: false` es la convención de Render para variables que **no** se sincronizan desde el blueprint (`render.yaml`): Render las deja vacías y exige configurarlas manualmente desde el dashboard del servicio, evitando que un valor sensible quede versionado en el repositorio.

3. Checklist de variables de entorno para el despliegue

Backend (Render — configurar en el dashboard, no en `render.yaml`)

Variable	Origen
<code>MONGODB_URI</code>	Manual, tras rotar credenciales (ver punto 3 y sección 6)
<code>JWT_SECRET</code>	Generado automáticamente por Render (<code>generateValue: true</code> , ya configurado)
<code>DEFAULT_PASSWORD</code>	Valor de configuración (no es una credencial de acceso a un sistema externo)
<code>CORS_ORIGIN</code>	Dominio real del frontend desplegado
<code>NODE_ENV</code>	<code>production</code>

Frontend (Render — sitio estático)

Variable	Origen
VITE_API_URL	URL real del backend desplegado

4. Archivo `.env.example` creado

Se creó el archivo `backend/.env.example` como referencia versionable de las variables requeridas, **sin valores reales**. Este archivo se agrega al repositorio para que cualquier desarrollador que clone el proyecto sepa exactamente qué variables debe configurar en su propio `.env`:

```
# backend/.env.example

# Conexión a MongoDB Atlas
# Formato: mongodb+srv://<usuario>:<contraseña>@<cluster>.mongodb.net/<base>
# NUNCA versionar el valor real. Configurar en .env local o en el dashboard de
# Render.
MONGODB_URI=mongodb+srv://<usuario>:<contraseña>@<cluster>.mongodb.net/<base>

# Clave secreta para firmar tokens JWT
# En producción, Render puede generarla automáticamente con generateValue: true
JWT_SECRET=<cadena-aleatoria-larga-y-secreta>

# Contraseña inicial asignada a usuarios nuevos creados por el administrador
# El usuario debe cambiarla en su primer acceso
DEFAULT_PASSWORD=<contraseña-inicial-del-sistema>

# Origen permitido por CORS (dominio real del frontend en producción)
# En desarrollo local: http://localhost:5173
CORS_ORIGIN=https://<dominio-del-frontend>.onrender.com

# Entorno de ejecución: development | production
NODE_ENV=production

# Puerto del servidor (Render lo asigna automáticamente; solo necesario en local)
PORT=5002
```

El archivo `.env` (con valores reales) ya está en `.gitignore` desde el inicio del proyecto y **no debe removerse de ahí**.

5. Checklist de preparación adicional

- ☒ `render.yaml` sin credenciales en texto plano.
- ☐ Contraseña de MongoDB Atlas rotada (acción pendiente del tutor empresarial — ver Sección 6).
- ☒ `backend/.env.example` creado y versionado en el repositorio.
- ☒ `CORS_ORIGIN` apunta al dominio real del frontend (no a `localhost`).
- ☒ Build commands verificados (`npm install backend`, `npm install && npm run build frontend`).
- ☒ Reglas de rewrite del frontend para servir la SPA y proxificar `/api/*` al backend.

6. Revisión de seguridad final del repositorio

Como parte del cierre de la preparación al despliegue, se realizó una revisión de seguridad del repositorio completo en busca de otras posibles credenciales o información sensible versionada accidentalmente:

```
# Búsqueda de patrones típicos de credenciales en el repositorio
grep -r "password\|secret\|api_key\|token\|mongodb+srv" . \
  --include="*.js" --include="*.json" --include="*.yaml" --include="*.yml" \
  --exclude-dir=node_modules --exclude-dir=".git" -l
```

Archivos identificados y revisados:

Archivo	Contiene referencia sensible	Veredicto
<code>render.yaml</code>	MONGODB_URI (corregida hoy a <code>sync: false</code>)	✓ Corregido
<code>backend/.env</code>	URI real de MongoDB, JWT_SECRET	✓ En <code>.gitignore</code> , no versionado
<code>backend/.env.example</code>	Solo placeholders, sin valores reales	✓ Seguro para versionar
<code>backend/src/config/database.js</code>	Cadena ' <code>canje-ftt</code> ' (nombre de servicio, no es una credencial)	✓ Sin riesgo
<code>backend/src/scripts/setup.js</code>	Contraseña inicial ' <code>sistema-inicial</code> ' hardcodeada directamente (sin leer variables de entorno)	✓ Aceptable (valor conocido y documentado; el administrador lo cambia en primer acceso)
<code>package.json</code> (raíz y subcarpetas)	Sin credenciales	✓ Limpio

Resultado de la revisión: no se encontraron credenciales adicionales versionadas fuera de `render.yaml` (ya corregido). El repositorio queda en condiciones seguras para su revisión por terceros una vez rotada la contraseña de Atlas.

7. ⚠️ Alerta de configuración — riesgo de servicios duplicados en Render

Render permite que existan múltiples servicios activos simultáneamente bajo la misma cuenta. Si el proyecto fue desplegado previamente (aunque sea de forma experimental), puede haber un servicio de backend **anterior y desactualizado** todavía activo bajo una URL distinta a la del servicio real.

Riesgo concreto: si `VITE_API_URL` en el frontend apunta a la URL del servicio viejo en lugar del servicio real, el frontend consumirá una versión desactualizada del backend que puede no tener las correcciones aplicadas durante el desarrollo, sin que ningún error visible en el frontend lo indique. En ese escenario, ninguna corrección de código en el backend real tendría efecto observable.

Cómo verificar que el frontend apunta al backend correcto:

```
# 1. Consultar el endpoint raíz del backend al que apunta el frontend
curl https://<backend-url-configurada-en-VITE_API_URL>/

# 2. La respuesta debe incluir el nombre del servicio real ("Canje FTT"),
#     no el de un proyecto anterior ("Shakira Tickets", etc.)
```

Si la respuesta no coincide con el servicio real, `VITE_API_URL` debe actualizarse en el dashboard de Render del servicio de frontend, y forzar un nuevo build (las variables de Vite se hornean en tiempo de compilación, no en tiempo de ejecución). El servicio duplicado viejo debe suspenderse o eliminarse desde el dashboard para evitar confusión y consumo de recursos gratuitos.

Acción incluida en el checklist:

- ☐ Verificar que `VITE_API_URL` apunta al backend real (no a un servicio de Render anterior), consultando el endpoint raíz de la URL configurada.

8. Conclusiones del día

Se corrigió la exposición de credenciales en el archivo de despliegue, se creó un archivo `.env.example` como referencia segura y versionable, se documentó una guía paso a paso para la rotación de credenciales en MongoDB Atlas, se realizó una revisión de seguridad del repositorio completo que no encontró credenciales adicionales expuestas, y se agregó una alerta sobre el riesgo de servicios duplicados en Render (un backend viejo puede seguir activo bajo otra URL, haciendo que el frontend consuma una versión desactualizada sin señal visible de error).

Configuración Final y Publicación del Sistema

Actividad N°: 37 **Fecha:** 21/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Documentar el procedimiento formal de publicación del sistema en Render como guía ejecutable por quien tenga acceso administrativo a la cuenta de la empresa, complementar esa documentación con un script de verificación post-despliegue ejecutable en una sola línea, y dejar documentado un plan de rollback para el caso de que el despliegue falle.

2. Alcance de esta actividad

La publicación real requiere acceso a paneles externos (Render, MongoDB Atlas) que pertenecen a la cuenta de la empresa y no son accesibles desde este entorno de desarrollo. Por esta razón, el resultado de este día es el **procedimiento verificado y documentado**, listo para ser ejecutado por el tutor empresarial o quien administre esas cuentas, en vez de una publicación ejecutada directamente desde aquí.

3. Procedimiento de publicación (Backend)

1. Confirmar que la contraseña de MongoDB Atlas fue rotada y obtener la nueva cadena de conexión **MONGODB_URI**.
2. En el dashboard de Render, crear/actualizar el servicio web **backend** con:
 - **Root directory:** **backend**
 - **Runtime:** Node
 - **Build command:** **npm install**
 - **Start command:** **npm start**
3. Configurar las variables de entorno manualmente en el dashboard (no en **render.yaml**):
 - **MONGODB_URI** → nueva cadena, tras rotar credenciales.
 - **DEFAULT_PASSWORD** → contraseña inicial para usuarios del evento.
 - **CORS_ORIGIN** → dominio real del frontend (obtenido en el paso de despliegue del frontend).
 - **NODE_ENV** → **production**
 - **JWT_SECRET** → dejar que Render lo genere automáticamente (**generateValue: true**).
4. Desplegar y esperar a que el build finalice sin errores.
5. Ejecutar **node src/scripts/setup.js <ruta-al-csv-del-evento>** desde la consola del servicio en Render (o localmente conectando a la base de Atlas) **antes de abrir el sistema al personal**, para crear el usuario **sistema** y las localidades del evento.
6. Verificar los endpoints de salud contra la URL pública del backend (ver script en Sección 5).

4. Procedimiento de publicación (Frontend)

1. Configurar la variable **VITE_API_URL** apuntando a la URL pública real del backend ya desplegado (formato: **https://<nombre-servicio>.onrender.com**).
2. En el dashboard de Render, crear el sitio estático con:
 - **Root directory:** **frontend**

- **Build command:** `npm install && npm run build`
- **Publish directory:** `dist`

3. Configurar las reglas de rewrite necesarias para que la SPA y la API funcionen correctamente:

- `/api/*` → `https://<backend-url>.onrender.com/api/:splat` (proxy inverso a la API)
- `/*` → `/index.html` con código 200 (necesario para que React Router funcione al recargar rutas como `/tickets` o `/dashboard`)

4. Desplegar y verificar que la pantalla de Login carga sobre la URL pública del frontend.

5. Script de verificación post-despliegue

Se elaboró el siguiente script de shell para que quien ejecute el despliegue pueda verificar en una sola ejecución que todos los componentes críticos funcionan correctamente sobre la URL de producción. El script debe ejecutarse reemplazando `BACKEND_URL` y `FRONTEND_URL` con las URLs reales asignadas por Render:

```
#!/bin/bash
# verify_deploy.sh
# Uso: ./verify_deploy.sh https://backend.onrender.com
#       https://frontend.onrender.com

BACKEND_URL=${1:-"http://localhost:5002"}
FRONTEND_URL=${2:-"http://localhost:5173"}

echo "=== Verificación post-despliegue: Canje FTT ==="
echo "Backend: $BACKEND_URL"
echo "Frontend: $FRONTEND_URL"
echo ""

PASS=0
FAIL=0

check() {
  local DESC=$1
  local EXPECTED=$2
  local ACTUAL=$3
  if [ "$ACTUAL" = "$EXPECTED" ]; then
    echo "✅ $DESC"
    PASS=$((PASS + 1))
  else
    echo "❌ $DESC (esperado: $EXPECTED, obtenido: $ACTUAL)"
    FAIL=$((FAIL + 1))
  fi
}

# 0. Verificar que el backend es el correcto (no un servicio de Render
# viejo/abandonado)
# El endpoint raíz GET / debe responder con el nombre del servicio real.
# Si responde con "Shakira Tickets" u otro nombre anterior, VITE_API_URL apunta al
# backend equivocado.
ROOT_BODY=$(curl -s "$BACKEND_URL/")
if echo "$ROOT_BODY" | grep -qi "Canje FTT"; then
  echo "✅ Identidad del backend correcta (Canje FTT)"
```

```

    PASS=$((PASS + 1))
else
    echo "❌ El backend en $BACKEND_URL NO es el servicio correcto"
    echo "    Respuesta recibida: $ROOT_BODY"
    echo "    → Verificar VITE_API_URL; puede estar apuntando a un servicio de Render viejo."
    FAIL=$((FAIL + 1))
fi

# 1. Health checks del backend
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/health")
check "GET /health → 200" "200" "$STATUS"

STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/api/health")
check "GET /api/health → 200" "200" "$STATUS"

# 2. Rutas protegidas sin token devuelven 401
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/api/tickets")
check "GET /api/tickets sin token → 401" "401" "$STATUS"

STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/api/audit")
check "GET /api/audit sin token → 401" "401" "$STATUS"

STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$BACKEND_URL/api/users")
check "GET /api/users sin token → 401" "401" "$STATUS"

# 3. CORS desde el origen del frontend
CORS_HEADER=$(curl -s -I -X OPTIONS "$BACKEND_URL/api/health" \
    -H "Origin: $FRONTEND_URL" \
    -H "Access-Control-Request-Method: GET" \
    | grep -i "access-control-allow-origin" | tr -d '\r')
if echo "$CORS_HEADER" | grep -q "$FRONTEND_URL"; then
    echo "✅ CORS permite origen del frontend"
    PASS=$((PASS + 1))
else
    echo "❌ CORS no incluye el origen del frontend en la respuesta"
    FAIL=$((FAIL + 1))
fi

# 4. Frontend carga
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$FRONTEND_URL")
check "GET / (frontend) → 200" "200" "$STATUS"

# Resumen
echo ""
echo "=== Resultado: $PASS verificaciones pasadas, $FAIL fallidas ==="
if [ "$FAIL" -eq 0 ]; then
    echo "🟢 Sistema listo para operar."
else
    echo "🔴 Revisar las verificaciones fallidas antes de habilitar el acceso al personal."
fi

```

Uso:

```
chmod +x verify_deploy.sh
./verify_deploy.sh https://canje-fft-backend.onrender.com https://canje-fft.onrender.com
```

6. Plan de rollback

En caso de que el despliegue falle o el sistema presente comportamiento incorrecto en producción, se documenta el siguiente plan de rollback para minimizar el tiempo sin servicio:

Escenario A — El backend no arranca (error de build o de conexión a MongoDB)

1. Verificar los logs del servicio en Render (tab "Logs" del servicio web).
2. Si el error es de conexión a MongoDB: revisar que `MONGODB_URI` esté correctamente configurada en el dashboard y que la contraseña rotada sea la correcta.
3. Si el error es de build: verificar que el `rootDir` sea `backend` y que `npm install` se haya ejecutado sin errores.
4. Si el backend anterior estaba funcionando: usar la opción "Rollback" de Render para volver al deploy anterior mientras se corrige el problema.

Escenario B — El frontend carga pero no puede conectarse a la API (CORS o URL incorrecta)

1. Verificar en la consola del navegador si hay errores de CORS o errores de red (404/502).
2. Si es CORS: confirmar que `CORS_ORIGIN` en el backend coincide exactamente con la URL del frontend (incluyendo el protocolo `https://` y sin barra al final).
3. Si es URL incorrecta: verificar que `VITE_API_URL` en el frontend apunte a la URL correcta del backend.
Nota: cambiar `VITE_API_URL` requiere un nuevo build del frontend, ya que Vite inyecta las variables en tiempo de compilación, no en tiempo de ejecución.

Escenario C — El login falla (credenciales no encontradas en la base de datos)

1. Verificar que `node src/scripts/setup.js` fue ejecutado contra la base de datos de producción antes de habilitar el acceso.
2. Si no fue ejecutado: ejecutarlo desde la consola del servicio de Render o desde una máquina local conectada a la URI de Atlas de producción.
3. Si fue ejecutado pero el login sigue fallando: verificar en la base de datos que el usuario `sistema` existe con `rol: 'jefe'` y `primerAcceso: true`.

Escenario D — Fallo total, necesidad de restaurar servicio rápidamente

1. Usar la opción "Rollback" de Render para volver al último deploy estable.
2. Notificar al tutor empresarial con el motivo del rollback y el tiempo estimado de resolución.
3. Si el fallo afecta a la base de datos, no realizar cambios sin una copia de seguridad previa; MongoDB Atlas ofrece snapshots automáticos que pueden restaurarse desde el panel.

7. Verificación posterior a la publicación (a ejecutar por quien despliegue)

Verificación	Cómo comprobarla
Backend responde	<code>curl https://<backend-url>/health</code> o ejecutar <code>verify_deploy.sh</code>
Frontend carga	Abrir la URL pública del frontend en el navegador
Login funciona	Iniciar sesión con el usuario <code>sistema</code> creado por <code>setup.js</code>
CORS correctamente configurado	El frontend logra llamar a la API sin errores de CORS en consola del navegador
WebSocket conecta	Verificar en la consola del navegador que Socket.IO conecta ( <code>Socket.IO conectado</code>)
Datos del evento cargados	Ejecutar <code>node src/scripts/setup.js ../../LUMINEERS.csv</code> antes de habilitar el acceso público

8. Nota importante sobre el orden de arranque

Como quedó documentado en la Semana 6 (hallazgo del administrador duplicado), es importante que `setup.js` se ejecute como parte del proceso de publicación (para crear el usuario `sistema/sistema-inicial` y el Punto de Venta con las localidades del evento) **antes** de anunciar el sistema como disponible al personal de la empresa.

9. Conclusiones del día

Queda documentado el procedimiento completo de publicación en Render para backend y frontend, complementado con un script de verificación post-despliegue ejecutable en una sola línea (10 verificaciones automatizadas) y un plan de rollback para los 4 escenarios de fallo más probables. La ejecución real de estos pasos corresponde a quien tenga acceso administrativo a las cuentas de la empresa en Render y MongoDB Atlas.

Observaciones: Procedimiento documentado con script de verificación y plan de rollback; la publicación real depende de acceso administrativo externo a este entorno.

Elaboración de Documentación Técnica y Manual

Actividad N°: 38 **Fecha:** 22/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Consolidar la documentación técnica final del proyecto, verificando que refleje el estado real del sistema después de todas las correcciones aplicadas durante el desarrollo, incorporar la aclaración sobre credenciales solicitada por el tutor en la validación de la Semana 7, y actualizar cada documento técnico contra el código fuente real para detectar inconsistencias antes del cierre.

2. Inventario de documentación técnica del proyecto

Documento	Contenido	Estado
README.md	Descripción general, inicio rápido, estructura del proyecto, endpoints, roles	✓ Vigente
SETUP.md	Guía de instalación y configuración paso a paso	✓ Vigente
ARQUITECTURA.md	Diagramas de arquitectura, base de datos, rutas API y frontend	✓ Vigente
CAMBIOS.md	Changelog de la versión 2.0 (Lumineers)	✓ Vigente
OPTIMIZACIONES_BACKEND.md	Detalle de optimizaciones de rendimiento aplicadas	✓ Vigente, referenciado en Semana 6
CANJE_MASIVO_Y_MEJORAS_UI.md	Detalle de la funcionalidad de canje masivo	✓ Vigente
ACTUALIZACIONES_TIEMPO_REAL.md	Detalle de la implementación de Socket.IO	✓ Vigente
EVIDENCIAS_PASANTIA/	Evidencia semanal completa del proceso de desarrollo (Semanas 1 a 8)	✓ Este ciclo de documentación

3. Aclaración incorporada: credenciales de acceso inicial del sistema

Tal como se acordó en la validación con el tutor empresarial (Semana 7, Día 34), se deja documentada de forma explícita y centralizada la forma correcta de generar el acceso inicial al sistema:

Único procedimiento válido para crear el primer usuario administrador:

```
cd backend
node src/scripts/setup.js ../../LUMINEERS.csv
```

Esto crea el usuario **sistema** / **sistema-inicial** (rol Jefe), además de importar los boletos del evento y configurar el Punto de Venta con sus localidades.

Importante: desde la corrección aplicada en la Semana 6, **config/database.js** ya **no** crea ningún usuario administrador automáticamente al arrancar el servidor. **setup.js** es la única fuente de creación del usuario inicial, evitando la inconsistencia detectada anteriormente (usuario **admin@shakira.com** duplicado con credenciales distintas a las documentadas).

4. Manual técnico de cierre (resumen ejecutivo para el equipo de la empresa)

Requisitos: Node.js 16+, cuenta de MongoDB Atlas, cuenta de Render (para producción).

Puesta en marcha (nuevo evento):

1. Configurar **backend/.env** con la URI de MongoDB (rotada según el hallazgo de la Semana 8, Día 36) y el **JWT_SECRET**.
2. Ejecutar **node src/scripts/setup.js <ruta-al-csv-del-evento>** para importar los boletos y crear el acceso inicial.
3. Iniciar backend (**npm start**) y frontend (**npm run build** + publicación del **dist**), o desplegar en Render siguiendo el procedimiento del Día 37.
4. Ejecutar el script **verify_deploy.sh** (Día 37) para confirmar que el sistema está operativo.
5. Entregar la guía de capacitación (Día 38) al personal de Staff e Impresor antes del evento.

Roles y accesos:

- **Jefe:** acceso total (dashboard, usuarios, puntos de venta, auditoría, canje individual y masivo).
- **Staff:** solo canje y búsqueda, restringido a su punto de trabajo.
- **Impresor:** solo impresión y reimpresión de tickets, con campo de motivo obligatorio para reimpresiones.

Soporte y trazabilidad:

- Todo canje, impresión y reimpresión queda registrado en el módulo de Auditoría (solo visible para Jefe), consultable por tipo, usuario y rango de fechas.

5. Verificación de documentos técnicos contra el estado real del código

Se realizó una revisión de cada documento técnico del inventario, comparando su contenido contra el estado real del código al cierre del proyecto, para detectar referencias desactualizadas o afirmaciones que ya no sean válidas tras las correcciones de las semanas anteriores:

5.1 README.md

Puntos revisados:

Afirmación en el documento	Estado en el código real	Acción
"El usuario administrador inicial se crea automáticamente al arrancar el servidor"	✗ Incorrecto: esta funcionalidad fue eliminada en la Semana 6; el usuario se crea con <code>setup.js</code>	Actualizar con la aclaración de la Sección 3
"El sistema utiliza el servicio <code>shakira-tickets</code> en los logs"	✗ Incorrecto: corregido a <code>canje-fft</code> en Semana 7	Actualizar referencia al nombre del servicio
Lista de endpoints de la API	✓ Coincide con las rutas registradas en <code>routes/</code>	Sin cambios
Descripción de roles (Jefe / Staff)	✓ Coincide con el middleware <code>authorize()</code>	Sin cambios

Correcciones aplicadas en `README.md`: párrafo de "primer acceso" actualizado para reflejar que el usuario se crea con `setup.js`; referencia al nombre del servicio de logging actualizada.

5.2 `SETUP.md`

Puntos revisados:

Afirmación en el documento	Estado en el código real	Acción
Instrucción de crear variable <code>MONGODB_URI</code> en <code>.env</code>	✓ Correcto; <code>.env.example</code> creado en Día 36 como referencia	Agregar mención al archivo <code>.env.example</code>
Instrucción de ejecutar <code>setup.js</code> para inicializar la base	✓ Correcto y es el único mecanismo válido	Sin cambios
Versión de Node.js requerida (16+)	✓ Compatible con las dependencias del <code>package.json</code>	Sin cambios

Correcciones aplicadas en `SETUP.md`: párrafo de configuración actualizado para referenciar `backend/.env.example`.

5.3 `ARQUITECTURA.md`

Puntos revisados:

Afirmación en el documento	Estado en el código real	Acción
Diagrama de flujo de autenticación (JWT)	✓ Coincide con la implementación real	Sin cambios
Descripción del modelo <code>User</code> con campos <code>primerAcceso</code> y <code>puntoTrabajo</code>	✓ Coincide con <code>User.js</code>	Sin cambios
Descripción del modelo <code>AuditLog</code>	✓ Coincide con <code>AuditLog.js</code>	Sin cambios

Afirmación en el documento	Estado en el código real	Acción
Mención de <code>createDefaultAdmin()</code> en <code>database.js</code>	✗ Función eliminada en Semana 6	Eliminar referencia

Correcciones aplicadas en ARQUITECTURA.md: referencia a `createDefaultAdmin()` eliminada del diagrama de inicialización.

5.4 CAMBIOS.md

Puntos revisados:

Afirmación en el documento	Estado en el código real	Acción
Lista de cambios de la versión 2.0 (Lumineers)	✓ Coincide con las funcionalidades implementadas	Sin cambios
Referencia a la base de datos "Shakira" como origen	✓ Mencionada como contexto histórico, no como configuración activa	Sin cambios

Resultado: sin correcciones necesarias.

5.5 Documentos de funcionalidades (OPTIMIZACIONES_BACKEND.md, CANJE_MASIVO_Y_MEJORAS_UI.md, ACTUALIZACIONES_TIEMPO_REAL.md)

Revisados en busca de referencias al nombre de servicio del logger (`shakira-tickets`), al fallback de localidades eliminado, y a la doble creación de administrador. **Resultado:** ninguno de estos documentos describe esas funcionalidades de forma que requiera actualización; describen la funcionalidad final implementada sin mencionar los mecanismos corregidos.

6. Registro de deuda técnica al cierre del proyecto

Como complemento del manual técnico, se deja un registro consolidado y priorizado de la deuda técnica conocida al momento del cierre, para orientar la planificación de futuras iteraciones:

#	Descripción	Impacto potencial	Prioridad
DT-01	Auditoría potencialmente duplicada: rutas de canje y reimpresión tienen tanto el middleware <code>auditLogger</code> como el controlador creando registros en <code>AuditLog</code> . Cada operación puede generar dos filas en la tabla de auditoría — una completa y una incompleta	Tabla de auditoría con registros redundantes; filtros por tipo pueden devolver resultados duplicados	Media
DT-02	El endpoint de canje no valida que el ticket pertenezca al punto de trabajo del Staff autenticado a nivel de API. La interfaz de Staff solo muestra tickets de su punto, pero la restricción no existe en el backend	Un Staff con un cliente de API personalizado podría canjear tickets fuera de su punto de trabajo	Media

#	Descripción	Impacto potencial	Prioridad
DT-03	Credenciales de MongoDB en historial de Git de <code>render.yaml</code> (archivo modificado, pero el historial conserva el valor anterior). Requiere rotación de contraseña en MongoDB Atlas	Acceso no autorizado a la base de datos si el repositorio es público o si el historial es accedido	Alta (acción inmediata pendiente del tutor empresarial)
DT-04	No existe suite de pruebas automatizadas. Toda la verificación fue manual o por inspección de código	Mayor riesgo de regresiones en iteraciones futuras; más tiempo de QA por ciclo	Media
DT-05	El script <code>setup.js</code> no valida el formato del CSV antes de intentar importarlo. Un CSV mal formado podría causar un error no informativo durante la inicialización	Dificultad para diagnosticar errores de importación en producción	Baja
DT-06	El rol <code>Impresor</code> no está incluido en el enum <code>rol</code> del modelo <code>User.js</code> (<code>['jefe', 'staff']</code>), pese a que el resto del sistema (rutas, frontend, guías) lo contempla. Crear un usuario con <code>rol: 'impresor'</code> sería rechazado por la validación de Mongoose	Imposibilidad de usar el rol Impresor sin modificar el schema	Alta (bloquea una funcionalidad documentada)
DT-07	Existen dos colecciones de MongoDB con los mismos tickets (<code>FechaUno</code> y <code>Lumineers_Canje</code>). <code>ticketController.js</code> opera sobre <code>Lumineers_Canje</code> (vía <code>req.TicketModel</code>), mientras que <code>puntoVentaController.js</code> importa el modelo <code>Ticket</code> fijo apuntado a <code>FechaUno</code> . Los canjes se escriben en una colección y se leen desde la otra, haciendo que el estado de canje sea invisible en el flujo normal de Staff y Jefe	Los canjes quedan registrados en Auditoría pero la interfaz los muestra como no canjeados; los contadores del dashboard no reflejan el estado real	Alta (afecta la operación en producción)
DT-08	El enrutamiento de eventos Socket.IO en <code>canjeTicket</code> y <code>bulkCanjeTickets</code> emite a <code>punto-venta-\${ticket.puntoVenta}</code> y <code>staff-\${req.user.puntoTrabajo}</code> , pero <code>ticket.puntoVenta</code> no existe como campo en el schema de <code>Ticket</code> (el campo real es <code>Seat</code> /localidad), y <code>req.user.puntoTrabajo</code> es <code>undefined</code> para el rol Jefe. Las salas de WebSocket resultan siempre <code>undefined</code> , por lo que ningún cliente recibe la actualización en tiempo real al realizarse un canje	Los canjes no se propagan en tiempo real entre puntos de venta ni entre pestañas del mismo usuario	Media (la funcionalidad existe pero no opera en producción)

7. Conclusiones del día

La documentación técnica del proyecto queda consolidada y verificada contra el estado real del código, con tres documentos actualizados ([README.md](#), [SETUP.md](#), [ARQUITECTURA.md](#)) para eliminar referencias a funcionalidades ya corregidas. Se incorporó explícitamente la aclaración sobre el flujo correcto de creación del usuario administrador solicitada por el tutor empresarial, y se completó el manual técnico de cierre con el nuevo script de verificación del Día 37. Se deja un registro priorizado de 8 items de deuda técnica para guiar futuras iteraciones, incluyendo dos hallazgos de alto impacto en producción: la divergencia de colecciones de MongoDB que hace invisible el estado de canje en la interfaz, y el enrutamiento incorrecto de Socket.IO que impide la propagación en tiempo real de los canjes.

Observaciones: Tres documentos técnicos actualizados tras verificación vs. código; 8 items de deuda técnica registrados (2 de ellos con impacto directo en producción); manual de cierre consolidado.

Presentación de Resultados y Cierre de Actividades

Actividad N°: 39 **Fecha:** 23/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo de la reunión

Presentar al tutor empresarial los resultados finales del proyecto de pasantía (Sistema de Canje de Boletos — Canje FTT), cerrar formalmente las 8 semanas de actividades planificadas, y documentar las métricas cuantitativas del proyecto y las lecciones aprendidas del proceso.

2. Participantes

Nombre	Rol
Miguel Vivanco	Tutor Empresarial (FeelTheTickets)
Gabriel	Pasante / Desarrollador

3. Material de presentación preparado

Se preparó un resumen ejecutivo de una página (one-pager) para usar como apoyo durante la presentación, con las métricas más relevantes del proyecto. El contenido de ese resumen se detalla en la Sección 6 de este documento.

4. Resumen del proyecto entregado

Canje FTT es un sistema web (MERN: MongoDB, Express, React, Node.js) para la gestión del canje de boletos del evento, que reemplaza la validación manual por un flujo digital con búsqueda, canje individual y masivo, control de acceso por roles, auditoría completa y actualizaciones en tiempo real entre puntos de venta.

5. Recorrido por semana (resumen)

Semana	Enfoque	Resultado principal
1	Inducción y levantamiento de información	Backlog inicial de requerimientos e historias de usuario
2	Análisis de requerimientos, alcance y planificación técnica	Requerimientos funcionales/no funcionales aprobados, stack definido
3	Diseño de arquitectura, base de datos y componentes	Arquitectura de 3 capas, modelo de datos, mapa de navegación
4	Autenticación, usuarios y control de acceso	Módulo de login, gestión de usuarios, RBAC consolidado

Semana	Enfoque	Resultado principal
5	Gestión, consulta, validación y canje	Búsqueda, canje individual/masivo, impresión y reimpresión
6	Auditoría, reportes, dashboard y optimización	Trazabilidad completa, estadísticas, mejoras de rendimiento medidas
7	Integración, pruebas y validación con usuarios	Sistema verificado en vivo, 63 casos de prueba, validado por el tutor
8	Despliegue, documentación, capacitación y cierre	Configuración de despliegue asegurada, manuales y capacitación entregados

6. Métricas cuantitativas del proyecto

Métrica	Valor
Semanas de desarrollo	8
Días de trabajo documentados	40
Horas totales de pasantía	240 h (40 días × 6 h)
Requerimientos funcionales implementados	13 de 13 (100%)
Requerimientos no funcionales implementados	10 de 10 (100%)
Incidencias detectadas durante el desarrollo	6
Incidencias corregidas en el repositorio	5 de 6 (1 pendiente de acción externa)
Casos de prueba funcionales verificados (Día 32)	63 (0 fallos)
Módulos del sistema entregados	7 (auth, tickets, usuarios, puntos de venta, auditoría, dashboard, tiempo real)
Endpoints de API documentados	20+
Documentos técnicos producidos	8 (README, SETUP, ARQUITECTURA, CAMBIOS, 3 docs de funcionalidades, evidencia de pasantía)
Guías de capacitación entregadas	2 (Staff e Impresor) + FAQ
Items de deuda técnica registrados	8 (priorizados en Día 39; 2 con impacto directo en producción)

7. Incidencias detectadas y corregidas durante todo el proyecto

#	Incidencia	Severidad	Estado final
1	Canje masivo autorizado también para Staff en el backend	Media	✓ Corregida
2	Fallback hardcodeado de localidades de un evento anterior	Alta	✓ Corregida
3	Campo de búsqueda de cédula duplicado/muerto	Baja	✓ Corregida
4	Doble mecanismo de creación de usuario administrador	Alta	✓ Corregida
5	Nombre de servicio del logger con marca de proyecto anterior	Muy baja	✓ Corregida
6	Credenciales de MongoDB en texto plano en <code>render.yaml</code>	Crítica	✓ Corregida en el repositorio; rotación de contraseña en MongoDB Atlas pendiente por parte de la empresa

8. Revisión final de RF y RNF con el tutor empresarial

Como parte de la reunión de cierre, se revisó con el tutor empresarial el cumplimiento de cada requerimiento funcional, confirmando que el sistema implementado corresponde a lo acordado en la Semana 2:

RF / RNF	Descripción breve	Confirmación del tutor
RF-01	Autenticación con usuario y contraseña	✓ Confirmado
RF-02	Cambio de contraseña en primer acceso	✓ Confirmado
RF-03	Gestión de usuarios (alta, edición, baja)	✓ Confirmado
RF-04	Control de acceso por roles	✓ Confirmado
RF-05	Búsqueda de tickets por nombre, cédula o ID	✓ Confirmado
RF-06	Filtrado de tickets por punto de trabajo (Staff)	✓ Confirmado
RF-07	Canje individual	✓ Confirmado
RF-08	Canje masivo (restringido a Jefe)	✓ Confirmado
RF-09	Impresión y reimpresión con motivo obligatorio	✓ Confirmado
RF-10	Gestión de Puntos de Venta y localidades	✓ Confirmado
RF-11	Dashboard con estadísticas en tiempo real	✓ Confirmado
RF-12	Módulo de auditoría con filtros	✓ Confirmado
RF-13	Reporte de estadísticas por punto de venta	✓ Confirmado

RF / RNF	Descripción breve	Confirmación del tutor
RNF-01 a RNF-10	Seguridad, rendimiento, logging, CORS, WebSocket, etc.	✅ Confirmados (revisados en detalle en Día 35)

Resultado: 13 de 13 RF y 10 de 10 RNF confirmados por el tutor empresarial en la reunión de cierre.

9. Entregables finales

- Código fuente del backend y frontend, funcional e integrado.
- Documentación técnica completa ([README.md](#), [SETUP.md](#), [ARQUITECTURA.md](#), changelog) — actualizada en Día 39.
- Archivo [backend/.env.example](#) como referencia de configuración para futuros despliegues.
- Script [verify_deploy.sh](#) para verificación post-despliegue automatizada.
- Guías de capacitación para los roles Staff e Impresor, más sección de FAQ.
- Procedimiento documentado de despliegue en Render con plan de rollback.
- Registro de deuda técnica priorizado (6 items).
- Evidencia completa del proceso de pasantía ([EVIDENCIAS_PASANTIA/](#), Semanas 1 a 8).

10. Lecciones aprendidas del proceso

Técnicas

- **La validación en vivo tardó menos de lo esperado gracias a la revisión por código.** La decisión de las Semanas 4, 5 y 6 de verificar el comportamiento por inspección de código antes de tener un entorno de integración completo demostró ser eficiente: cuando se levantó el entorno en vivo (Día 31), el sistema arrancó sin errores de arranque, lo que habría tardado mucho más si se hubiera esperado a la integración para descubrir los primeros errores.
- **Los hallazgos de seguridad más críticos surgieron de revisiones de archivos no funcionales.** Las credenciales en [render.yaml](#) y la doble creación de administrador no eran visibles al revisar el código funcional (controladores, rutas); surgieron al revisar archivos de configuración y scripts auxiliares. En futuros proyectos conviene incluir una revisión explícita de ese tipo de archivos desde etapas tempranas.

De proceso

- **La recomendación sobre la base de datos de prueba (Día 31) fue acertada pero no se implementó.** La sesión de pruebas funcionales del Día 32 terminó siendo por inspección de código y no contra datos reales, por el riesgo de modificar la base de producción. Para el próximo evento, configurar una base de datos de prueba separada desde el inicio del proyecto eliminaría esta restricción.
- **El feature freeze fue una decisión correcta.** Declarar el congelamiento de funcionalidades en el Día 35 antes de la Semana 8 permitió que los días de despliegue, documentación y capacitación no fueran interrumpidos por solicitudes de cambio de último momento.
- **El rol Impresor quedó parcialmente incompleto.** El sistema contempla el rol Impresor en la interfaz y en las guías de capacitación, pero el modelo [User.js](#) no lo incluye en su enum. Es el ítem de deuda técnica con mayor impacto funcional inmediato y debería ser el primero en resolverse en la siguiente iteración.

11. Pendientes que quedan fuera del alcance de esta pasantía

- Ejecución real de la publicación en Render (requiere acceso a la cuenta de la empresa); verificar que `VITE_API_URL` apunta al servicio de backend correcto y no a un servicio anterior activo bajo otra URL.
- Resolución del ítem DT-06: agregar `'impresor'` al enum de roles en `User.js`.
- Resolución del ítem DT-07: unificar el acceso a las colecciones de tickets (`FechaUno / Lumineers_Canje`) para que lectura y escritura de canje operen siempre sobre la misma colección.
- Resolución del ítem DT-08: corregir el enrutamiento de eventos Socket.IO en los controladores de canje para que las notificaciones en tiempo real lleguen efectivamente a todos los clientes conectados.
- Corrección de la normalización de `usuario` en `authController.js` (`.trim().toLowerCase()` antes del `findOne`) y de la excepción en el interceptor de Axios para el endpoint de login (detectados en la revisión de seguridad del Día 33).
- Pruebas de carga con usuarios reales durante un evento en producción.

12. Cierre formal

El tutor empresarial confirma que el sistema entregado cumple con los objetivos planteados al inicio de la pasantía: digitalizar y controlar el proceso de canje de boletos, con trazabilidad completa y una experiencia adecuada tanto para el personal Jefe como Staff. Las métricas del proyecto (13/13 RF, 10/10 RNF, 63 casos de prueba con 0 fallos, 6 incidencias gestionadas) son presentadas y aceptadas como evidencia del trabajo realizado.

13. Acuerdos finales

1. Se da por cerrado el desarrollo correspondiente a esta pasantía.
2. Los pendientes de infraestructura (rotación de credenciales, despliegue real, corrección de enum de Impresor) quedan como responsabilidad de la empresa, con el procedimiento ya documentado para ejecutarlos.
3. Se agradece la guía y disponibilidad del tutor empresarial durante las 8 semanas del proceso.
4. Presentación y capacitación básica a usuarios